Containers, Modern DevOps, and Migration Strategy

CSC-535 Cloud DevOps

Muhammad Tausif

Department of Computer Science
QAU

Lecture Roadmap

Understanding Docker Images

Docker images act as the **blueprint for Docker containers**.

A Docker image defines:

- Application code
- Required libraries
- Dependencies
- Runtime environment

Key Idea:
An image contains everything needed for an application to run consistently across different environments.

What Does a Docker Image Contain?

A Docker image is built through a series of steps defined in a **Dockerfile**. Typical steps include:

- Starting from a base OS image
- Installing system packages
- Adding application dependencies
- Copying source code
- Running build commands

Each step creates a new **layer** in the image.

Docker Image Storage

Docker images are stored in a **container registry**.
Examples include:

- Docker Hub
- AWS Elastic Container Registry
- Google Container Registry

Developers can:

- Pull images from registry
- Push images to registry
- Share images across systems

Layered Filesystem in Docker

Docker images use a **layered filesystem**.
Instead of one large block of storage:

- Images consist of multiple layers
- Each layer represents a change
- Layers are stacked on top of each other

Each Dockerfile command creates a new layer.

How Layers Are Created

Every instruction in a Dockerfile generates a new image layer.
Example workflow:

- Pull base image (Ubuntu)
- Install Python
- Add application dependencies
- Copy application code

Each step modifies the previous layer and creates a new one.

Why Docker Uses Layers

Advantages of layered images:

- Faster image building
- Efficient storage
- Reduced network transfer
- Reuse of common base images

If two images share the same base layer, Docker reuses it instead of duplicating it.

Containers and Writable Layer

When a container is created from an image:

- Image layers remain **read-only**
- Docker adds a **writable layer** on top

All runtime changes occur in this writable layer.
Examples:

- Creating files
- Modifying configurations
- Writing logs

Container Layer Structure

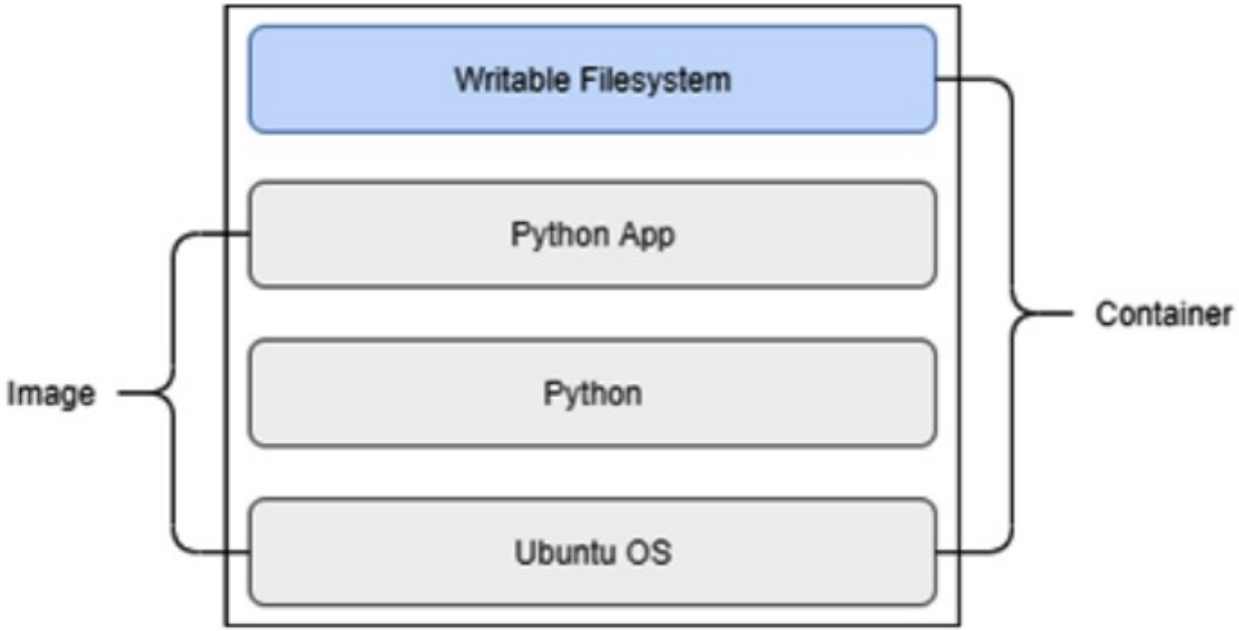

Figure 3.2 – Container layers

Structure:

- Ubuntu OS layer
- Python runtime layer
- Application layer
- Writable container layer

Summary: Docker Images and Layers

- Docker images are blueprints for containers
- Images contain application code and dependencies
- Docker uses layered filesystem architecture
- Each Dockerfile instruction creates a new layer
- Containers add a writable layer on top of images

Layering improves efficiency, speed, and storage utilization.

Docker Image History

Docker allows us to inspect how an image was built by viewing its **image history**.

Image history shows:

- The layers that compose an image
- The commands used to create each layer
- When each layer was created
- The size of each layer

This helps developers understand how a container image is constructed.

Viewing Docker Image History

To inspect the history of a Docker image, use the command:

```
docker history <image_name>
```

Example:

```
docker history bharamicrosystems/python-flask-redis
```

This command displays all the layers that make up the image along with the commands used to build them.

Understanding Image History Output

The output of `docker history` typically includes:

- **IMAGE ID** – Unique identifier of the layer
- **CREATED** – Time when the layer was created
- **CREATED BY** – Command used to create the layer
- **SIZE** – Disk space used by that layer

Some layers may show `<missing>` as the image ID. This indicates intermediate layers created during image build.

Why Image Layers Matter

Each instruction in a Dockerfile modifies the filesystem and creates a new image layer.

Important observations:

- Some layers occupy disk space (e.g., installing packages)
- Some layers occupy no space (e.g., CMD or ENV instructions)
- Every layer represents the difference from the previous layer

Key Idea:
Docker images are built as a sequence of incremental filesystem changes.

Summary: Docker Image History

- Docker images consist of multiple layers.
- The command `docker history` reveals these layers.
- Each Dockerfile instruction creates a new layer.
- Some layers modify the filesystem while others only define metadata.

Understanding image history helps optimize Docker images and reduce their size.

Understanding Dockerfiles

A Dockerfile is a text file that contains a sequence of instructions used to build a Docker image.

Each instruction in the Dockerfile is called a **directive**.

Dockerfiles allow developers to:

- Define application environment
- Install required dependencies
- Configure services
- Automate image creation

Dockerfiles enable reproducible and automated container builds.

Example Dockerfile

A simple Dockerfile example for running an NGINX container:

```
FROM ubuntu:xenial
RUN apt update && apt install -y curl
RUN apt update && apt install -y nginx
CMD ["nginx", "-g", "daemon off;"]
```

This Dockerfile builds an NGINX container starting from an Ubuntu base image.

The FROM Directive

The **FROM** directive defines the base image for the container.

Example:

```
FROM ubuntu:xenial
```

This means:

- The container will use Ubuntu as the base operating system
- All subsequent layers are built on top of this base image

Choosing the correct base image is critical for security, size, and performance.

The RUN Directive

The **RUN** directive executes commands during image build.

Example:

```
RUN apt update && apt install -y nginx
```

Characteristics:

- Executes commands inside the image
- Creates a new image layer
- Installs packages and dependencies

Multiple commands can be combined using `&&`.

Why apt update Appears Multiple Times

Docker images are built in layers.

Each layer is independent and does not automatically inherit temporary state from previous layers.

Therefore:

- `apt update` must be executed before installing packages
- Otherwise package indexes may become outdated

Each layer should contain everything required for that step.

The CMD Directive

The **CMD** directive defines the default command executed when the container starts.

Example:

```
CMD ["nginx", "-g", "daemon off;"]
```

Purpose:

- Runs the application inside the container
- Keeps the container process active

For long-running services like NGINX, the process must run in the foreground.

Summary: Dockerfile Directives

- Dockerfiles automate image creation.
- The **FROM** directive defines the base image.
- The **RUN** directive executes build-time commands.
- The **CMD** directive defines the container startup command.
- Docker builds images layer by layer.

Dockerfiles are essential for reproducible container environments.

Using ENTRYPOINT Instead of CMD

In Dockerfiles, the **ENTRYPOINT** directive can be used instead of CMD.

Both directives define commands that run when the container starts, but they behave differently.

Key difference:

- CMD provides default arguments
- ENTRYPOINT defines the main executable

ENTRYPOINT is often used when you want the container to behave like a fixed application.

How CMD Works

Example:

```
CMD ["nginx", "-g", "daemon off;"]
```

Characteristics:

- Defines the default command executed when the container starts
- Can be overridden when running the container
- Used for providing default runtime behavior

Example override:

```
docker run image_name bash
```

This replaces the CMD instruction.

How ENTRYPOINT Works

Example:

```
ENTRYPOINT ["nginx", "-g"]
```

If a user runs:

```
docker run image_name "daemon off;"
```

Docker combines them:

```
nginx -g daemon off;
```

Important:

- Commands provided during container execution are appended
- ENTRYPOINT is not easily overridden

ENTRYPOINT vs CMD

CMD	ENTRYPOINT
Defines default command	Defines main executable
Can be overridden easily	Harder to override
Used for default arguments	Used for fixed container behavior
More flexible	More secure and controlled

Best Practice:
Use ENTRYPOINT when you want to enforce a specific container behavior.

Are RUN and CMD the Same?

RUN and CMD serve different purposes in a Dockerfile.

- **RUN** executes commands during image build.
- **CMD** defines the command executed when the container starts.

Key difference:

- RUN modifies the **image filesystem**.
- CMD runs in the **container runtime**.

Multiple RUN statements can exist, but only the **last CMD** is used.

RUN vs CMD

RUN	CMD
Executed during image build	Executed when container starts
Creates new image layer	Runs inside container
Used to install software	Defines default command
Cannot be overridden easily	Can be overridden at run-time

Building Our First Container Image

Docker images are built using the command:

```
docker build -t <image-name>:<version> <build-context>
```

Example:

```
docker build -t user/nginx-hello-world .
```

- Docker reads the Dockerfile
- Executes each instruction sequentially
- Creates image layers

Docker Image Naming Structure

Docker image names follow this structure:
¡registry-url¿/¡account-name¿/¡image-name¿:¡version¿
Example:
`docker.io/user/nginx-hello-world:latest`
Components:

- Registry URL – image repository
- Account name – owner of image
- Image name – container image
- Version – image tag

Running a Container

Run container from the built image:

```
docker run -d -p 80:80 user/nginx-hello-world
```

Options:

- **-d** run container in background
- **-p 80:80** map host port to container port

Verify container:

```
docker ps
```

Testing the Container

To verify that the container is running:

```
curl localhost
```

This command fetches the webpage served by NGINX inside the container.

Expected output:

Welcome to nginx!

Customizing the Container

Instead of the default NGINX page, we can add a custom page.
Create a file:
`index.html`
Example content:
Hello World! This is my first docker image!

Updated Dockerfile

Example Dockerfile:

```
FROM ubuntu:xenial
RUN apt update && apt install -y curl
RUN apt update && apt install -y nginx
WORKDIR /var/www/html
ADD index.html ./
CMD ["nginx","-g","daemon off;"]
```

WORKDIR Directive

WORKDIR sets the working directory inside the container.
Example:

```
WORKDIR /var/www/html
```

Benefits:

- Commands execute relative to this directory
- Simplifies file operations
- Ensures consistent application location

ADD Directive

The ADD directive copies files into the container.
Example:

```
ADD index.html ./
```

Capabilities:

- Copy local files
- Extract archives automatically
- Download files from URLs

Often **COPY** is preferred for simplicity.

Docker Build Cache

Docker uses a layered architecture.
Benefits:

- Faster builds
- Reuse previously built layers
- Only rebuild modified steps

Best practice:
Add application code **after installing dependencies**.

Restarting Containers

Remove old container:

```
docker rm <container-id> -f
```

Run new container:

```
docker run -d -p 80:80 user/nginx-hello-world
```

Verify:

```
docker ps
```

EXPOSE Directive

EXPOSE specifies which port the container application uses.
Example:

```
EXPOSE 80
```

Purpose:

- Document application port
- Improve container configuration clarity

Health Monitoring with HEALTHCHECK

Docker allows monitoring container health.
Example:

```
HEALTHCHECK CMD curl -f localhost
```

Optional parameters:

- interval – check interval
- timeout – check timeout
- retries – failure attempts

This helps detect unhealthy containers.

Container Health Status

Docker container health states:

- starting
- healthy
- unhealthy

Health monitoring improves reliability and service monitoring in production systems.

Building and Managing Docker Images

Docker images are created using Dockerfiles.
During the build process Docker:

- Reads instructions sequentially
- Creates layers for each instruction
- Reuses cached layers when possible

Efficient Docker image design is important for faster builds, smaller images, and better CI/CD performance.

Docker Image Best Practices

Recommended practices when writing Dockerfiles:

- Place layers that change less frequently first
- Place frequently changing layers later
- Combine commands to reduce number of layers
- Include only required files in container images

These practices improve build performance and reduce image size.

Layer Ordering Strategy

Docker uses layer caching during builds.
Example strategy:

- Install dependencies first
- Copy application code later

Reason:

- Dependencies change rarely
- Source code changes frequently

This prevents unnecessary rebuilding of earlier layers.

Combining Commands in Dockerfile

Multiple commands should be combined to reduce image layers.
Example:

```
RUN apt update && apt install -y curl nginx
```

Advantages:

- Fewer layers
- Smaller image size
- Faster container startup

Single-Stage Builds

A single-stage build compiles and runs an application within a single container image.
Example application:
A simple Go program that prints:
Hello, World!
This concept works for any programming language.

Example Go Application

Example Go program:

```
package main
import "fmt"
func main()
{
fmt.Println("Hello, World!")
}
```

This application simply prints a message when executed.

Dockerfile for Go Application

Example Dockerfile:

```
FROM golang:1.12.4
WORKDIR /tmp
COPY app.go .
RUN GOOS=linux go build -o app
CMD ["./app"]
```

Steps performed:

- Pull Go base image
- Copy source code
- Compile program
- Run executable

Building the Image

Build the Docker image using:

```
docker build -t user/go-hello-world:single_stage .
```

Docker will:

- Download base image
- Execute Dockerfile instructions
- Create new container image

After build completion, the image becomes available locally.

Running the Container

Run the built container:

```
docker run user/go-hello-world:single_stage
```

Expected output:

Hello, World!

To view images:

```
docker images
```

Problem with Single-Stage Builds

Single-stage builds often create large container images.
Example:

- Image size may exceed hundreds of MB
- Includes unnecessary build tools
- Contains full development environment

This is inefficient for production environments.

Why Images Become Large

The Go base image includes:

- Go compiler
- Build tools
- Libraries
- Development environment

However, the application only needs the compiled binary to run. This leads to unnecessarily large images.

Next Step: Multi-Stage Builds

Docker solves this problem using **multi-stage builds**.
Concept:

- Build application in one stage
- Copy compiled binary to a lightweight image

Benefits:

- Smaller container images
- Improved security
- Faster deployments

Multi-Stage Docker Builds

Multi-stage builds allow Docker images to be built in multiple stages.

- Separate build environment from runtime environment
- Reduce final image size
- Improve efficiency and security

Key idea:

- Use one stage to build the application
- Use another lightweight stage to run the application

Example Multi-Stage Dockerfile

Example Dockerfile:

```
FROM golang:1.12.4 AS build
WORKDIR /tmp
COPY app.go .
RUN GOOS=linux go build -o app

FROM alpine:3.12.1
WORKDIR /tmp
COPY --from=build /tmp/app .
CMD ["./app"]
```

Understanding the First Stage

First stage:

```
FROM golang:1.12.4 AS build
```

Purpose:

- Uses Golang image with compiler
- Builds the application binary
- Produces executable file

The stage is named **build** using the AS keyword.

Understanding the Second Stage

Second stage:

`FROM alpine:3.12.1`

Purpose:

- Uses a lightweight Alpine Linux image
- Copies only the compiled binary
- Excludes build tools and dependencies

This significantly reduces the final image size.

Copying Files Between Stages

Docker allows copying artifacts from previous stages.
Example:

```
COPY --from=build /tmp/app .
```

Meaning:

- Copy compiled binary from build stage
- Place it in the runtime container

Only the necessary files are transferred.

Building the Multi-Stage Image

Build the image:

```
docker build -t user/go-hello-world:multi_stage .
```

Docker performs the following steps:

- Build application in the first stage
- Copy binary to lightweight runtime image
- Produce final optimized container image

Running the Container

Run the container:

```
docker run user/go-hello-world:multi_stage
```

Expected output:

Hello, World!

Even though the output is the same, the container image size is much smaller.

Image Size Comparison

Single-stage build:

- Image size: 784 MB

Multi-stage build:

- Image size: 7.57 MB

This represents a **100x reduction** in image size.

Best Practices for Multi-Stage Builds

Guidelines:

- Use multi-stage builds whenever possible
- Keep runtime images minimal
- Remove build dependencies
- Use lightweight images like Alpine

This approach improves performance, security, and deployment speed.

Managing Docker Images

In modern DevOps environments:

- Docker images are usually built on developer machines
- Or built inside CI/CD pipelines
- Images are stored in container registries
- Then deployed to staging and production environments

Containers may run:

- Directly on Docker
- Through orchestrators such as Kubernetes

Using Docker Images Efficiently

Docker automatically pulls images when needed.
Example:
`docker run nginx`
If the image is not present locally:

- Docker pulls it from the registry
- Stores it locally

Once downloaded, Docker reuses the local copy.

Pulling Images Manually

To explicitly download an image:

```
docker pull nginx
```

This command:

- Downloads image layers
- Stores them locally
- Makes the image ready to run

Docker pulls only missing layers.

Running Containers from Images

Once an image is available locally:

`docker run nginx`

Docker launches a container immediately.

No additional downloading is required unless the image is updated.

Why Avoid Using the Latest Tag

Using **latest** can create problems.
Reasons:

- Version changes may break deployments
- Difficult to roll back to previous versions
- Reduces deployment reproducibility

Best practice:
Use explicit version tags.
Example:
`nginx:1.25`

Benefits of Versioned Images

Version tags help:

- Maintain stable production deployments
- Roll back faulty updates
- Track application versions
- Maintain reproducible environments

Production systems should avoid the **latest** tag.

Removing Docker Images

Images can be removed using:

```
docker rmi nginx
```

If the image is being used by a running container, Docker will return an error.

Reason:

Containers depend on the image layers.

Force Removing Images

To remove an image forcefully:

```
docker rmi -f nginx
```

Warning:

- This stops and removes containers using the image
- Should be used carefully

Always verify containers before removing images.

Authenticating with Docker Hub

Before pushing images to Docker Hub:

`docker login`

This command authenticates the user with Docker Hub.

After authentication, images can be pushed to the registry.

Pushing Images to Docker Hub

Push an image:

```
docker push user/nginx-hello-world:latest
```

Docker will:

- Upload image layers
- Skip layers already present
- Store the image in the registry

Pushing Multiple Image Tags

If an image has multiple tags:

```
docker push user/go-hello-world
```

Docker pushes all associated tags.

This simplifies image version management.

Dangling Images

Dangling images are unused layers created during builds.
They appear when:

- Images are rebuilt
- Old layers are no longer referenced

These layers consume disk space unnecessarily.

Cleaning Up Images

Remove unused layers using:

`docker images prune`

Benefits:

- Reclaims disk space
- Cleans up old image layers
- Keeps Docker environment organized

Regular cleanup is recommended in production systems.

Flattening Docker Images

Docker images use a layered filesystem.
Advantages of layers:

- Efficient storage
- Faster builds
- Layer reuse across images

However, in some situations an image with fewer layers may perform better.
Flattening merges layers into a single layer.

When Flattening is Used

Flattening is not a common practice.
It is used only when:

- Image size must be minimized
- Layer complexity becomes high
- Performance improvements are required

Most of the time Docker's layered filesystem already provides good performance.

Steps to Flatten a Docker Image

Flattening a Docker image requires three steps:

1. Run a container from the image
2. Export the container filesystem
3. Import the exported filesystem as a new image

These steps merge all layers into a single layer.

Running a Container

Start a container from the image:

```
docker run -d --name nginx user/nginx-hello-world:latest
```

This container will be used to export the filesystem.

Exporting the Container

Export the container filesystem:

```
docker export nginx > nginx-flat.tar
```

This command creates a TAR archive containing the full filesystem.

The exported file contains all merged layers.

Importing as a New Image

Create a flattened image using:

```
cat nginx-flat.tar | docker import -
user/nginx-hello-world:flat
```

This creates a new Docker image with only one layer.

Verifying the Flattened Image

Check available images:

```
docker images
```

Example result:

- Original image: 231 MB
- Flattened image: 184 MB

Flattened images may occupy less space.

Image History Comparison

View image layers:

```
docker history user/nginx-hello-world:latest
```

Original image:

- Multiple layers
- Each Dockerfile instruction creates a layer

Flattened image:

- Only one layer

Should You Flatten Images?

Flattening images has disadvantages:

- Removes layer reuse benefits
- Increases storage if multiple images share layers
- Makes debugging harder

Better alternatives:

- Use multi-stage builds
- Use lightweight base images (Alpine)
- Combine RUN commands

Flattening should be used only when necessary.

Optimizing Containers with Distroless Images

Distroless images are minimal container images designed to run applications securely.
They contain only:

- Application binary
- Required runtime libraries
- Minimal operating system components

They remove unnecessary components such as:

- Package managers
- Shell environments
- Debugging tools

Why Optimize Containers?

Container optimization focuses on three major factors:

- Performance
- Security
- Cost

Smaller container images provide better efficiency across all three dimensions.

Performance Benefits

Large images consume more resources.
Each container launch requires:

- Downloading image layers
- Disk I/O operations
- Network bandwidth

Smaller images:

- Start faster
- Transfer faster across networks
- Improve deployment speed

Security Benefits

Traditional Linux images include many utilities.
Examples:

- Shells (bash)
- Package managers (apt)
- Debug tools

These tools increase the attack surface.
Distroless images remove them, which improves security.

Cost Benefits

Smaller container images reduce operational costs.
Benefits include:

- Reduced storage usage
- Faster deployment
- Lower network bandwidth consumption
- Higher container density per machine

This leads to long-term infrastructure savings.

Distroless Image Concept

Distroless containers contain only the application and required runtime libraries.
They do not include:

- Shell environments
- Package managers
- Unnecessary utilities

This results in a minimal container footprint.
Google maintains official distroless images.

Example Distroless Dockerfile

Example Dockerfile:

```
FROM golang:1.12.4 AS build
WORKDIR /tmp
COPY app.go .
RUN GOOS=linux go build -o app

FROM gcr.io/distroless/base
WORKDIR /tmp
COPY --from=build /tmp/app .
CMD ["./app"]
```

Building the Distroless Image

Build the container image:

```
docker build -t user/go-hello-world:distroless .
```

Docker will:

- Build the application in the first stage
- Copy binary into distroless runtime image
- Produce a minimal container image

Running the Distroless Container

Run the container:

```
docker run user/go-hello-world:distroless
```

Expected output:

Hello, World!

The container runs successfully without a full Linux distribution.

Image Size Comparison

Example sizes:

- Single-stage build: 784 MB
- Multi-stage build: 7.57 MB
- Distroless image: 18.9 MB

Distroless images provide a secure and optimized runtime environment.

Understanding Docker Registries

A Docker Registry is a server-side application used to store and distribute Docker images.

- Acts as a central storage for container images
- Allows pushing and pulling images
- Enables sharing images across environments

Docker registries help teams manage container images efficiently.

Docker Repositories

A Docker registry contains repositories.
A repository stores multiple versions of the same image.
Example:

- nginx:1.24
- nginx:1.25
- nginx:latest

Each version is identified using a tag.

Docker Hub

Docker Hub is the default public Docker registry.
Features:

- Public image hosting
- Community container images
- Automated builds
- Version control using tags

Example images available on Docker Hub:

- nginx
- ubuntu
- mysql

Private Docker Registries

Some container images must remain private.
Options for private registries:

- Self-hosted Docker Registry
- Cloud registries (AWS ECR, GCR, Azure ACR)
- Private Docker Hub repositories

Private registries allow organizations to securely store proprietary images.

Hosting Your Own Docker Registry

Docker provides an official registry image.
Start a local registry:

```
docker run -d -p 80:5000 --restart=always
--name registry registry:2
```

This launches a private Docker registry server.

Tagging Images for a Registry

Before pushing an image to a registry, it must be tagged.
Example:

```
docker tag user/nginx-hello-world:latest
localhost/user/nginx-hello-world:latest
```

Tagging tells Docker where the image should be pushed.

Pushing Images to the Registry

Push the tagged image:

```
docker push localhost/user/nginx-hello-world:latest
```

Docker uploads the image layers to the registry.

Layers already present in the registry are skipped.

Persistent Storage for Registries

Registries should store images in persistent storage.
Without persistent storage:

- Images will be lost after container restart

Volumes should be mounted to store registry data.

Adding Authentication

Authentication protects the registry from unauthorized access.
Create authentication directory:
`mkdir -p /mnt/registry/auth`
Generate password file using:
`htpasswd`
Users must authenticate before pushing or pulling images.

Enabling TLS Security

Registry communication should be encrypted.
Generate TLS certificates using:
`openssl`
Certificates provide:

- Secure image transfers
- Protection from man-in-the-middle attacks

Running a Secure Registry

Example secure registry configuration:

```
docker run -d -p 443:443
-v /mnt/registry/certs:/certs
-v /mnt/registry/auth:/auth
registry:2
```

This enables authentication and encrypted communication.

Logging Into the Registry

Authenticate using:

`docker login https://localhost`

After successful login, images can be pushed securely.

Other Public Container Registries

Apart from running your own Docker registry, several cloud providers offer managed container registries.
Advantages of managed registries:

- High availability
- Scalability
- Built-in security
- Integration with cloud infrastructure

These registries simplify container image management in cloud environments.

Amazon Elastic Container Registry (ECR)

Amazon Elastic Container Registry (ECR) is AWS's container registry service.

Features:

- Highly available and scalable
- Supports private and public repositories
- Integrates with AWS IAM for authentication
- Works with AWS services such as ECS and EKS

Users only pay for storage and network transfer usage.

Google Container Registry (GCR)

Google Container Registry (GCR) is Google's container registry solution. Key characteristics:

- Integrated with Google Cloud Platform
- Uses Google Cloud Storage for image storage
- Supports private and public repositories
- High performance and reliability

GCR is commonly used with Kubernetes clusters on Google Cloud.

Azure Container Registry (ACR)

Azure Container Registry (ACR) is Microsoft's container registry service. Capabilities:

- Fully managed registry service
- Geo-replication support
- Secure private repositories
- Integration with Azure Kubernetes Service (AKS)

ACR also stores Helm charts and other deployment artifacts.

Other Registry Solutions

Additional registry platforms include:

- Oracle Cloud Infrastructure Registry
- CoreOS Quay
- Sonatype Nexus Repository
- JFrog Artifactory

These solutions provide:

- Private repositories
- Security scanning
- Authentication systems
- Integration with CI/CD pipelines

Summary: Docker Registries

Key points:

- Docker registries store container images
- Docker Hub is the default public registry
- Organizations can host private registries
- Images must be tagged before pushing
- Authentication and TLS improve security

Docker registries are essential for CI/CD pipelines and container distribution.

Other Public Container Registries

Apart from running your own Docker registry, several cloud providers offer managed container registries.
Advantages of managed registries:

- High availability
- Scalability
- Built-in security
- Integration with cloud infrastructure

These registries simplify container image management in cloud environments.

Amazon Elastic Container Registry (ECR)

Amazon Elastic Container Registry (ECR) is AWS's container registry service.

Features:

- Highly available and scalable
- Supports private and public repositories
- Integrates with AWS IAM for authentication
- Works with AWS services such as ECS and EKS

Users only pay for storage and network transfer usage.

Google Container Registry (GCR)

Google Container Registry (GCR) is Google's container registry solution. Key characteristics:

- Integrated with Google Cloud Platform
- Uses Google Cloud Storage for image storage
- Supports private and public repositories
- High performance and reliability

GCR is commonly used with Kubernetes clusters on Google Cloud.

Azure Container Registry (ACR)

Azure Container Registry (ACR) is Microsoft's container registry service. Capabilities:

- Fully managed registry service
- Geo-replication support
- Secure private repositories
- Integration with Azure Kubernetes Service (AKS)

ACR also stores Helm charts and other deployment artifacts.

Other Registry Solutions

Additional registry platforms include:

- Oracle Cloud Infrastructure Registry
- CoreOS Quay
- Sonatype Nexus Repository
- JFrog Artifactory

These solutions provide:

- Private repositories
- Security scanning
- Authentication systems
- Integration with CI/CD pipelines

Docker Chapter Summary

In this chapter we explored the core concepts of Docker from a hands-on and DevOps perspective.
Key topics included:

- Docker images and layered filesystem
- Dockerfiles and their directives
- Building and running container images
- Managing Docker images efficiently

These concepts form the foundation of container-based deployment.

Advanced Docker Concepts Covered

We also studied several advanced container optimization techniques.

- Multi-stage Docker builds
- Flattening Docker images
- Distroless container images
- Container image optimization strategies

These approaches help create:

- Smaller images
- Faster deployments
- More secure containers

Container Distribution and Next Steps

We discussed how container images are distributed using Docker registries. Topics included:

- Docker Hub
- Private Docker registries
- Cloud registries (AWS ECR, GCR, Azure ACR)
- Image authentication and secure distribution

Next Topic:
Container orchestration using **Kubernetes**.